

Intrusion Detection System in IoT System

A PROJECT REPORT

Submitted by

Ranjan Mishra (231B258)

Rishabh Jain (231B264)

Rashmi Dubey (231B260)

Under the guidance of: Dr. Amit Kumar Srivastava



May-2026

Submitted in partial fulfillment for the award of the degree

of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING**

**Department of Computer Science & Engineering
JAYPEE UNIVERSITY OF ENGINEERING & TECHNOLOGY,
AB ROAD, RAGHOGARH, DT. GUNA-473226 MP, INDIA**

Table Of Content

Title Page	i
Table Of Content	ii
Declaration by Student	iv
Certificate	v
Acknowledgement	vi
Abstract	vii
Chapter 1 INTRODUCTION	6
Chapter 2 LITERATURE REVIEW	7
Chapter 3 METHODOLOGY	9
3.1 Overview	
3.2 Dataset Description	
3.2.1 NSL-KDD Dataset	
3.2.2 Attack Category Grouping	
3.3 Data Preprocessing	
3.3.1 Categorical Encoding	
3.3.2 Feature Scaling	
3.3.3 Stratified Train-Test Split	
3.4 Feature Engineering and Selection	
3.4.1 Engineered Features	
3.4.2 Mutual Information-Based Feature Selection	
3.5 Model Training	
3.5.1 Extra Trees Classifier — Motivation and Configuration	
3.5.2 Grid Search Hyperparameter Optimisation	

3.5.3 SMOTE for Class Imbalance	
3.6 Deep Learning Baseline — MLP	
3.6.1 Architecture	
3.6.2 Training and Optuna Hyperparameter Tuning	
3.7 Multi-Model Comparison	
3.8 Adversarial Robustness Evaluation	
3.8.1 Motivation and Threat Model	
3.8.2 Surrogate Network for Gradient Attacks	
3.8.3 Attack Methods	
3.8.4 Results and Robustness Metrics	
3.9 Model Serialisation and Deployment Artifacts	
3.10 Chapter Summary	
Chapter 4 HARDWARE AND SYSTEM ARCHITECTURE	23
4. Hardware	
4.1 ESP32-C5	
4.2 RFID-RC225	
4.3 Raspberry Pi 5 Model B	
4.4 ARCHITECTURE	
4.4.1 ESP32-C5 connection with RFID-RC225	
4.4.2 MQTT Connection with Raspberry Pi 5 Model B	
4.5 Implementation	
4.5.1 ESP32-C5 Code	
4.5.2 UI and Detection code	
4.5.3 MQTT command	
4.6 Testing	
4.6.1 Terminal Testing at Backend	
4.6.2 Telegram Notification and Chatbot Testing	
Chapter 5 CONCLUSION	52
Chapter 6 REFERENCES	54

DECLARATION

We declare that the work reported in the B.Tech 6th semester project entitled “Intrusion Detection System in IoT System”, submitted in partial fulfillment for the award of the B.Tech (CSE) degree at Jaypee University of Engineering and Technology, Guna, is my original work. To the best of my knowledge and belief, it does not infringe upon any intellectual property rights or copyrights. In case of any violation, I will be solely responsible.

Ranjan Mishra (231B258)

Rishabh Jain (231B264)

Rashmi Dubey (231B260)

Department of Computer Science and Engineering
Jaypee University of Engineering and Technology
Guna, M.P., India

Date: 05/05/2026



JAYPEE UNIVERSITY OF ENGINEERING & TECHNOLOGY

Accredited with Grade-A+ by NAAC & Approved U/S 2(f) of the UGC Act, 1956

A.B. Road, Raghogarh, District Guna (MP), India, Pin-473226

Phone: 07544 267051, 267310-14,

Fax: 07544 267011

Website: www.juet.ac.in

CERTIFICATE

This is to certify that the work titled "Intrusion Detection System in IoT System" submitted by Ranjan Mishra (231B258), Rishabh Jain (231B264), and Rashmi Dubey (231B260) in partial fulfilment for the award of the degree of B.Tech of Jaypee University of Engineering & Technology, Guna has been carried out under my supervision. To the best of my knowledge and belief, there is no infringement of intellectual property rights or copyright. This work has not been submitted, either partially or wholly, to any other University or Institute for the award of this or any other degree or diploma. In the event of any violation, the concerned student will be solely responsible.

Signature of Supervisor

(Dr. Amit Kumar Srivastava)

Department of Computer Science and Engineering

Jaypee University of Engineering and Technology

Guna, M.P., India

Date: 05/05/2026

ACKNOWLEDGEMENT

We have invested a lot of time and efforts in completion of this project, but this project would not have been possible without the kind support of various individuals and organizations. We here by extend our sincerest gratitude to all of them. We are highly indebted to our mentor, **Dr. Amit Kumar Srivastava**. We would like to thank him for his constant guidance and supervision. We would like to thank the JUET for providing us with such and supportive and innovative environment and facilities and lending us their support whenever required. We would also like to thank the whole organization for providing us with such opportunities.

Finally, we all would like to thank and congratulate our fellow project members on their hard work and dedication.

Thanking you

Ranjan Mishra (231B258)

Rishabh Jain (231B264)

Rashmi Dubey (231B260)

Date: 05/05/2026

ABSTRACT

The rapid growth of Internet of Things (IoT) devices has created major security challenges due to limited device capabilities and the need for real-time threat detection. Traditional intrusion detection systems (IDS) are often unsuitable for IoT environments because they require high computational resources and lack integration with hardware-level monitoring. This project addresses these challenges by developing a cost-effective and fully integrated IoT-based IDS that combines hardware, communication, machine learning, and alerting mechanisms.

The system's physical layer is built using the ESP32-C5 microcontroller integrated with an RC522 RFID module for access control. It continuously scans RFID cards, extracts unique IDs, and classifies access attempts as authorized or suspicious. The system also detects abnormal patterns such as repeated attempts or replay attacks using embedded monitoring logic, ensuring early detection of potential intrusions at the edge itself.

Communication between devices is handled using the MQTT protocol, a lightweight publish-subscribe messaging system optimized for IoT. The ESP32-C5 transmits real-time access data as JSON messages to an MQTT broker hosted on a Raspberry Pi 5 Model B. The Raspberry Pi acts as the central processing unit, responsible for managing communication, processing incoming data, and executing machine learning models. The intelligence layer is powered by an Extra Trees classifier trained on the NSL-KDD dataset, a standard benchmark for intrusion detection. The model development pipeline includes data preprocessing, feature engineering, feature selection, and hyperparameter tuning. Techniques such as SMOTE are used to handle class imbalance across multiple attack categories. The optimized model achieves high performance, with 99.55% accuracy and strong detection capability for minority attack classes, outperforming other models like Random Forest, XGBoost, and neural networks.

To ensure robustness, the system is evaluated against multiple adversarial attacks, including both white-box and black-box methods. Due to its tree-based structure, the model demonstrates strong resistance to gradient-based attacks, maintaining high accuracy under most conditions. While decision-based attacks like Boundary Attack show some impact, mitigation strategies such as ensemble methods are identified.

The system also includes a Telegram-based alerting interface that allows administrators to receive real-time notifications, monitor activity, and control access remotely. Overall, the project demonstrates that a low-cost, scalable, and integrated IoT IDS can achieve near-production-level performance, making it suitable for real-world smart environments.

1. Introduction

The rapid advancement of the Internet of Things (IoT) has transformed the way devices interact, communicate, and operate in modern environments. IoT systems are now widely deployed in areas such as smart homes, industrial automation, healthcare monitoring, transportation, and security systems. These systems consist of interconnected devices that continuously exchange data to provide automation, efficiency, and real-time decision-making. However, the increasing number of connected devices has also introduced significant security vulnerabilities. Due to limited computational resources, heterogeneous architectures, and large-scale deployments, IoT systems are highly susceptible to cyber threats such as unauthorized access, replay attacks, data interception, and device spoofing.

Traditional security mechanisms designed for conventional networks are often not suitable for IoT environments, as they require higher computational power and may not support real-time processing. As a result, there is a growing need for lightweight, efficient, and scalable intrusion detection systems (IDS) that can operate effectively within IoT constraints while ensuring system security and reliability.

This project proposes an IoT-based Intrusion Detection System that combines hardware-based identification, wireless communication, and intelligent monitoring to detect and prevent unauthorized access. The system is built using the ESP32-C5 microcontroller, the RC522 RFID module, and the Raspberry Pi 5, forming a complete end-to-end IoT architecture. The ESP32-C5 acts as the primary sensing and processing unit, interfacing with the RFID module to read Unique Identification Numbers (UIDs) from RFID cards. These UIDs represent user identity and are used for authentication purposes.

The communication between devices is established using the MQTT (Message Queuing Telemetry Transport) protocol, which is a lightweight and efficient messaging protocol specifically designed for IoT applications. The ESP32-C5 publishes RFID scan data to an MQTT topic, while the Raspberry Pi 5 acts as a central hub (broker/client) that receives, processes, and distributes the data to other components in the system. This publish-subscribe model reduces network overhead and enables scalable communication between multiple devices.

A Python-based backend system processes the incoming data and implements intrusion detection logic. The system compares scanned RFID UIDs with a predefined list of authorized users. If a valid UID is detected, the system marks the access as “OK”; otherwise, it generates an “ALERT” indicating a potential intrusion. Additionally, the system monitors repeated access attempts and identifies abnormal patterns such as replay attacks by maintaining counters for each UID. When suspicious behaviour exceeds a predefined threshold, the system can trigger alerts or automatically block the user.

To enhance usability and real-time monitoring, the system integrates a Telegram-based notification and control interface. This allows administrators to receive instant alerts, view access history, monitor connected devices, and perform actions such as blocking or unblocking users remotely. The inclusion of WiFi device monitoring further strengthens the system by detecting unknown devices connected to the network.

Overall, this project provides a comprehensive, cost-effective, and scalable solution for intrusion detection in IoT environments. By combining RFID-based authentication, MQTT-based communication, and real-time alert mechanisms, the system ensures secure access control, efficient data transmission, and proactive threat detection. The proposed approach bridges the gap between hardware-level security and intelligent software-based analysis, making it suitable for a wide range of applications such as smart homes, offices, and industrial security systems.

2.Literature Review

The security of the Internet of Things (IoT) has emerged as a major research domain due to the rapid expansion of connected devices and the inherent limitations of embedded systems [1]. IoT devices often operate with constrained computational power, memory, and energy resources, making the implementation of traditional security mechanisms challenging. Early work by Anthi et al. (2019) highlighted the importance of designing supervised intrusion detection systems (IDS) tailored specifically for smart home environments, where device behavior profiling plays a critical role in identifying anomalies and malicious activities [2]. These initial approaches laid the groundwork for behavior-based security monitoring in IoT ecosystems.

As IoT deployments became more complex, research shifted toward intelligent and adaptive detection mechanisms. Boughaci et al. (2024) introduced machine learning-based frameworks capable of handling high-dimensional IoT data and detecting complex attack patterns using ensemble learning techniques [3]. These approaches significantly improved detection accuracy and scalability compared to traditional rule-based systems. In parallel, comprehensive surveys by Boughaci and Al-Taharwa (2023) and Abdulkareem et al. (2024) emphasized that IoT networks are highly vulnerable to evolving cyber threats, including distributed denial-of-service (DDoS) attacks, spoofing, and data exfiltration [4]. Their findings suggest that hybrid detection models, which combine signature-based and anomaly-based approaches, consistently outperform standalone techniques in dynamic environments.

The emergence of adversarial machine learning has introduced new challenges to IoT security systems. Attackers can manipulate input data to deceive machine learning models, leading to misclassification and reduced detection effectiveness [5]. This has prompted the development of robust and resilient IDS frameworks capable of withstanding adversarial conditions. Alongside detection strategies, researchers have also explored the underlying communication technologies that enable IoT systems. Protocols such as Wi-Fi, Bluetooth, Zigbee, and cellular networks form the backbone of IoT connectivity, enabling seamless communication between billions of devices [6]. However, these protocols also introduce potential vulnerabilities that can be exploited if not properly secured.

Among communication protocols, the Message Queuing Telemetry Transport (MQTT) protocol has gained widespread adoption due to its lightweight design and efficiency in low-bandwidth environments [7]. MQTT operates on a publish-subscribe model, making it suitable for machine-to-machine (M2M) communication. Studies have demonstrated that MQTT-based systems can be vulnerable to attacks such as unauthorized publishing, topic hijacking, and denial-of-service if proper authentication and encryption are not implemented [8]. Practical implementations often utilize brokers like Mosquitto and client libraries such as Eclipse Paho to facilitate reliable communication between edge devices and centralized systems [9].

At the hardware level, IoT security also depends on reliable identification and authentication mechanisms. The MFRC522 RFID module, which complies with ISO/IEC 14443 standards, is widely used for contactless identification in access control systems [10]. While RFID technology provides convenience and efficiency, it is also susceptible to cloning, replay attacks, and unauthorized scanning if not combined with secure authentication methods. Therefore, integrating RFID systems with intelligent monitoring and anomaly detection is essential for enhancing physical security.

Edge computing has further transformed IoT architectures by enabling real-time data processing closer to the source. Devices such as the Raspberry Pi 5 Model B offer significant computational capabilities, including multi-core processors and high memory capacity, allowing them to function as

local processing hubs for IoT systems [11]. These edge devices can run machine learning models for real-time intrusion detection, reducing latency and dependency on cloud infrastructure. Similarly, modern microcontrollers like the ESP32-C5 integrate advanced features such as dual-band Wi-Fi 6, Bluetooth Low Energy, and RISC-V processing cores, enabling efficient and secure edge sensing [12].

Hardware-level security features have become increasingly important in protecting IoT devices from firmware-level attacks. Mechanisms such as secure boot, flash encryption, and hardware-based key storage ensure the integrity and authenticity of device firmware [13]. These features are particularly critical in large-scale deployments such as smart cities and industrial IoT systems, where compromised devices can have severe consequences. Recent studies have also explored scalable machine learning approaches for anomaly detection in both smart home and industrial environments, demonstrating that intelligent systems can identify subtle attack patterns that traditional methods may overlook [14].

Despite these advancements, IoT security remains a complex challenge due to the heterogeneity of devices and network configurations. Effective security solutions must integrate multiple layers, including device authentication, secure communication, and intelligent threat detection [15]. In critical sectors such as healthcare, energy, and smart grids, security requirements are even more stringent. Researchers have proposed advanced authentication mechanisms, including delegation-based models and biometric-based remote user authentication, to ensure secure access control [16].

The evolving threat landscape includes sophisticated attacks targeting not only individual devices but also entire networks and infrastructures. Smart grids, for example, are vulnerable to coordinated cyberattacks that can disrupt power distribution and compromise critical services [17]. To address these challenges, research has expanded into cloud-based intrusion detection systems that leverage neural networks and deep learning for improved accuracy and scalability [18]. Additionally, federated learning approaches have been proposed to enhance privacy and enable collaborative model training across distributed IoT devices without sharing raw data [19].

Machine learning models used in IDS have also been continuously refined. Techniques such as Random Forest, Support Vector Machines, and Gradient Boosting have been widely studied for their effectiveness in intrusion detection [20]. More recently, deep learning models, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), have been applied to capture temporal and spatial patterns in network traffic [21]. However, these models often require significant computational resources, limiting their applicability in resource-constrained IoT environments.[23]

Adversarial robustness remains an active area of investigation, with studies examining the impact of various attack strategies, including gradient-based, score-based, and decision-based attacks [26]. Tree-based models have been shown to exhibit inherent resistance to certain adversarial attacks due to their non-differentiable structure, making them suitable for security-critical applications [27]. However, no single model is entirely immune, and ensemble approaches are often recommended to enhance robustness and reliability [28].

Overall, the literature demonstrates that effective IoT security requires a holistic approach that integrates hardware, communication protocols, machine learning, and robust authentication mechanisms. Continuous advancements in edge computing, intelligent detection models, and secure communication frameworks are essential to address the evolving challenges in IoT environments [29].

3. Methodology

3.1 Overview

This chapter presents a detailed account of the methodology adopted to design, develop, and evaluate a machine learning-based Intrusion Detection System (IDS) tailored for IoT network environments. The proposed system leverages the NSL-KDD benchmark dataset and employs a systematic pipeline encompassing data preprocessing, feature engineering, model training, hyperparameter optimisation, class-imbalance handling, and adversarial robustness evaluation.

The methodology is structured around the following major stages:

- Dataset acquisition and initial exploration
- Data preprocessing and label normalisation
- Feature engineering and mutual information-based selection
- Baseline model training — Extra Trees Classifier
- Hyperparameter optimisation via Grid Search Cross-Validation
- Class imbalance mitigation using SMOTE
- Deep learning baseline — Multi-Layer Perceptron (PyTorch)
- Multi-model comparison: Random Forest, XGBoost, MLP, Extra Trees
- Adversarial robustness evaluation using seven distinct attack strategies

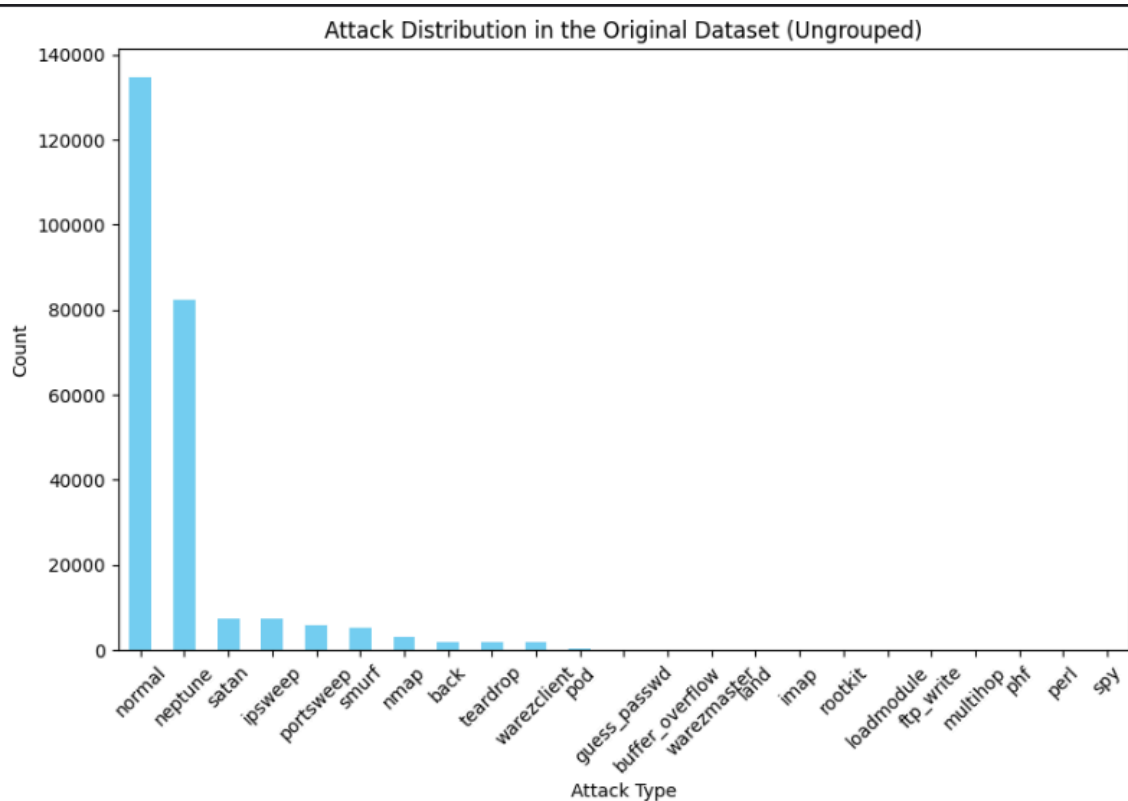
Each stage is described in detail in the sections that follow. All experiments are conducted in a Kaggle Python environment using scikit-learn, PyTorch, imbalanced-learn, and the Adversarial Robustness Toolbox (ART).

3.2 Dataset Description

3.2.1 NSL-KDD Dataset

The NSL-KDD dataset is a refined version of the widely-used KDD Cup 1999 dataset, specifically designed to address several inherent problems of its predecessor, including redundant records in training and test sets that biased classifier performance. It is the most widely adopted benchmark for evaluating network IDS models due to its balanced representation of attack categories and the absence of duplicate records that plagued KDD 1999.

Each record in the dataset describes a single network connection and is composed of 41 features alongside a target label indicating either a benign connection (Normal) or one of approximately 39 distinct attack types. The dataset is partitioned into two standard splits: KDDTrain+ and KDDTest+. In this work, both splits are concatenated into a unified corpus before applying a stratified 80/20 train-test division, ensuring that the evaluation is not sensitive to the original partitioning strategy.



The 41 features span four broad categories: basic connection features (duration, protocol type, service, flag, byte counts), content features (login failures, root shell, file creations), traffic features (error rates, same-service counts), and host-based features (destination host statistics). Full feature names are listed in the preprocessing section.

3.2.2 Attack Category Grouping

The 39 raw attack subtypes are consolidated into four semantically meaningful high-level categories, reducing label sparsity and aligning the classification task with standard network security taxonomy:

- DOS (Denial of Service): back, land, pod, smurf, teardrop, neptune, mailbomb, processtable, udpstorm, apache2, worm — flood-based attacks that overwhelm network resources
- Probe: ipsweep, satan, nmap, portsweep, mscan, saint — reconnaissance attacks that scan for vulnerabilities
- R2L (Remote to Local): warezclient, imap, spy, multihop, guess_passwd, ftp_write, warezmaster, phf, xlock, xsnoop, snmpguess, snmpgetattack, httptunnel, sendmail, named — unauthorised access from a remote machine
- U2R (User to Root): perl, rootkit, buffer_overflow, loadmodule, sqlattack, xterm, ps — privilege escalation attacks

The resulting five-class problem — DOS, Probe, R2L, U2R, and Normal — exhibits severe class imbalance. DOS and Normal instances collectively account for over 95% of records, while U2R contributes fewer than 0.1%. This imbalance is explicitly addressed in Section 3.5.

3.3 Data Preprocessing

3.3.1 Categorical Encoding

Three features in the NSL-KDD dataset are categorical: `protocol_type` (tcp, udp, icmp), `service` (70+ distinct network service types), and `flag` (11 connection states). These are transformed using Label Encoding, which maps each unique string value to an integer code. A dedicated `LabelEncoder` object is fitted and stored for each feature to enable consistent inverse transformation during inference.

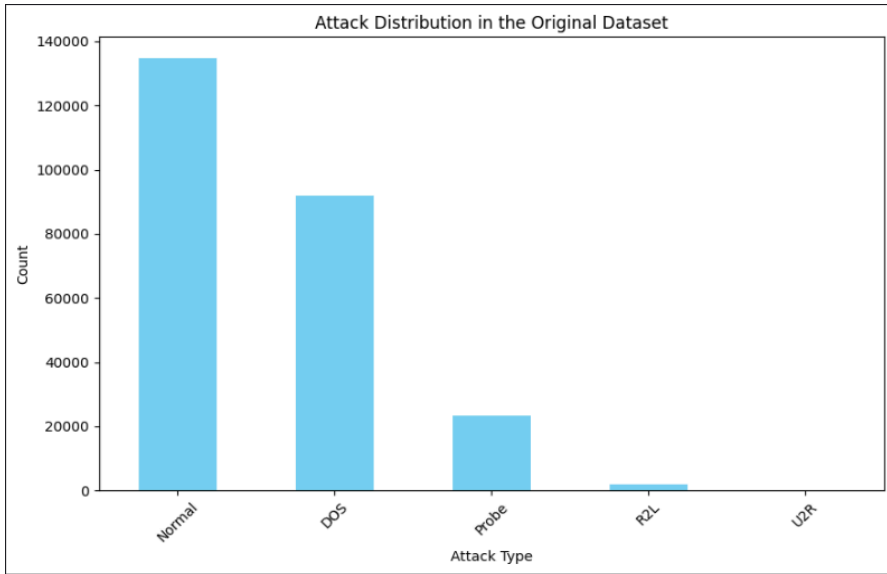
Label Encoding is preferred over One-Hot Encoding for tree-based models because ensemble trees can natively exploit ordinal relationships without the dimensionality explosion that one-hot encoding would introduce — encoding the service feature alone would add over 70 binary columns.

3.3.2 Feature Scaling

`StandardScaler` is fitted exclusively on the training partition and applied to both splits to prevent data leakage. Each feature is centred at zero mean and unit variance. While tree-based models are inherently scale-invariant, scaling is applied for consistency with the MLP and to ensure valid clamping during adversarial perturbation experiments.

3.3.3 Stratified Train-Test Split

An 80/20 stratified train-test split is applied with `random_state=42`. The `stratify=y` parameter ensures that the proportion of each attack category is preserved in both partitions, which is critical given the strong class imbalance. Without stratification, minority classes such as U2R could be entirely absent from the test set by chance.



3.4 Feature Engineering and Selection

3.4.1 Engineered Features

Three new interaction features are derived from existing raw attributes to capture non-linear relationships between traffic volume metrics that are not directly represented in the original feature set:

- $\text{byte_ratio} = \text{src_bytes} / (\text{dst_bytes} + 1)$: Captures directional traffic asymmetry. A high ratio indicates outbound-dominant traffic, characteristic of data exfiltration (R2L) or one-sided flooding (DOS). The denominator is shifted by 1 to avoid division-by-zero.
- $\text{byte_diff} = \text{src_bytes} - \text{dst_bytes}$: An absolute measure of traffic asymmetry, complementing `byte_ratio` with magnitude sensitivity rather than scale-invariant ratio.
- $\text{duration_src_bytes} = \text{duration} \times \text{src_bytes}$: An interaction term that amplifies signals for long-lived, high-volume connections, which are characteristic of persistent worm-based and flooding intrusions.

These features increase discriminative capacity with negligible computational cost, requiring only arithmetic operations at both training and inference time. Following their addition, Extra Trees accuracy improved from 99.3% to 99.5%.

3.4.2 Mutual Information-Based Feature Selection

The augmented 44-feature set is subjected to mutual information-based selection using `SelectKBest(mutual_info_classif, k=30)`. Mutual information quantifies the statistical dependence between each feature and the multi-class target, capturing both linear and non-linear associations without imposing distributional assumptions. The top 30 features are retained, as summarised in Table 3.1.

Category	Features	Description
Basic	duration, src_bytes, dst_bytes	Raw traffic volume and timing metrics
Protocol	protocol_type, service, flag	Label-encoded categorical attributes
Traffic Stats	count, srv_count, error_rates	Connection-level statistical aggregates
Host-level	dst_host_count, dst_host_srv_*	Destination host behaviour statistics
Engineered	byte_ratio, byte_diff, duration_src_bytes	Derived non-linear interaction features

Table 3.1: Feature categories retained after mutual information-based selection (top 30 of 44).

The choice of $k=30$ balances information retention against model complexity. Given the dataset size (~150K records), the marginal computational cost of additional features is negligible, and aggressive pruning risks discarding subtle signals relevant to rare attack classes such as U2R.

3.5 Model Training

3.5.1 Extra Trees Classifier — Motivation and Configuration

The Extra Trees (Extremely Randomised Trees) Classifier is selected as the primary model based on three distinguishing properties that are particularly advantageous for IoT IDS applications:

- Variance reduction: Greater tree diversity through random split thresholds reduces overfitting on high-dimensional tabular data
- Training efficiency: Eliminating the exhaustive threshold search yields significantly faster training compared to standard Random Forest
- Adversarial robustness: Non-differentiable outputs provide inherent resistance to gradient-based adversarial attacks

A baseline Extra Trees model (`n_estimators=100`, `random_state=42`, `n_jobs=-1`) establishes a performance floor of 99.3% accuracy on the raw feature set, confirming the model's suitability for this task prior to any optimisation.

3.5.2 Grid Search Hyperparameter Optimisation

GridSearchCV with 3-fold stratified cross-validation is applied to determine the optimal Extra Trees configuration. The search space spans eight candidate configurations across three hyperparameters:

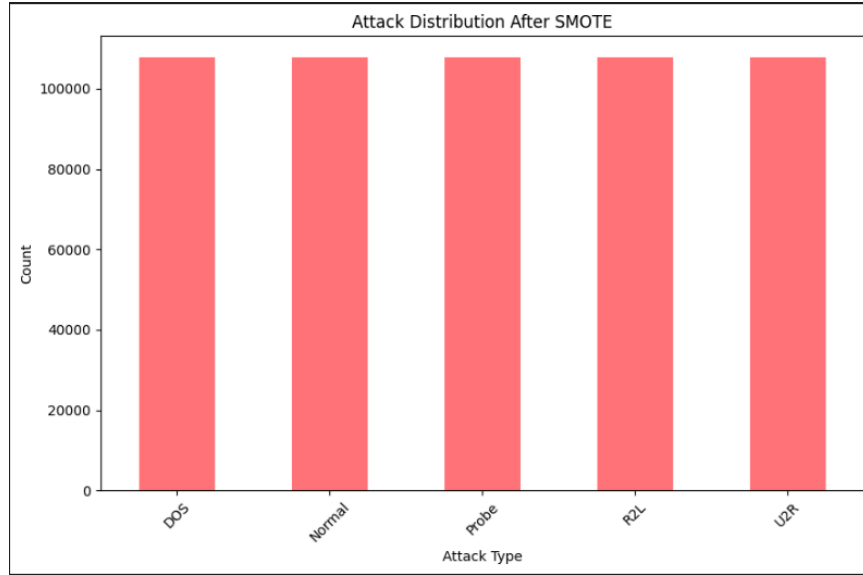
- `n_estimators`: {50, 100} — number of trees in the ensemble
- `max_depth`: {None, 10} — maximum depth of each tree
- `min_samples_split`: {2, 5} — minimum samples required to split an internal node

The best configuration is selected based on cross-validated accuracy and applied to all downstream evaluations and adversarial experiments. Models are evaluated with `n_jobs=-1` to exploit all available CPU cores.

3.5.3 SMOTE for Class Imbalance

Synthetic Minority Over-sampling Technique (SMOTE) is applied exclusively to the training partition with `sampling_strategy='auto'`, which up-samples all minority classes to match the majority class count. SMOTE generates synthetic samples by linearly interpolating between existing minority instances and their k-nearest neighbours in feature space, expanding the decision boundary without mere duplication.

The impact is substantial: R2L recall improved from 0.59 to 0.95 (+36 pp) and U2R recall from 0.33 to 0.58 (+25 pp). The macro-average F1 score improved from 0.85 to 0.92. Critically, overall accuracy remained at 99.55%, confirming that the gains in minority class detection came at negligible cost to majority class performance.



3.6 Deep Learning Baseline — MLP

3.6.1 Architecture

A Multilayer Perceptron (MLP) is implemented in PyTorch as a complementary deep learning baseline and as the surrogate model for gradient-based adversarial attacks. The final tuned architecture — DeeperMLP — comprises four fully connected layers with ReLU activations and Dropout regularisation:

- Layer 1: Linear(input_dim $\rightarrow$ 256), Dropout(p=0.2), ReLU
- Layer 2: Linear(256 $\rightarrow$ 128), Dropout(p=0.2), ReLU
- Layer 3: Linear(128 $\rightarrow$ 64), Dropout(p=0.2), ReLU
- Output: Linear(64 $\rightarrow$ num_classes) — no activation

Dropout layers with p=0.2 are inserted after each hidden layer to regularise the network. The output layer applies no activation; PyTorch's CrossEntropyLoss internally applies log-softmax, avoiding numerical instability. Gradient clipping (max_norm=1.0) is applied during training to prevent exploding gradients.

3.6.2 Training and Optuna Hyperparameter Tuning

The MLP is trained using AdamW optimiser with weighted cross-entropy loss, where class weights are computed via sklearn's compute_class_weight(balanced). Optuna's Tree-structured Parzen Estimator (TPE) sampler is used to search the following hyperparameter space across 5 trials:

- Hidden layer sizes: hidden1 in {64, 128, 192, 256}, hidden2 in {32, 64, 96, 128}
- Learning rate: log-uniform in [1e-5, 1e-2]
- Batch size: {32, 64, 128}
- Weight decay: log-uniform in [1e-6, 1e-3]

The best configuration identified is: `hidden1=64`, `hidden2=96`, `lr=1.42×10-4`, `batch_size=32`, `weight_decay=2.27×10-6`. This checkpoint is saved as `mlp_best_checkpoint.pth`. The final model is retrained for 15 epochs with these hyperparameters and achieves approximately 99% test accuracy.

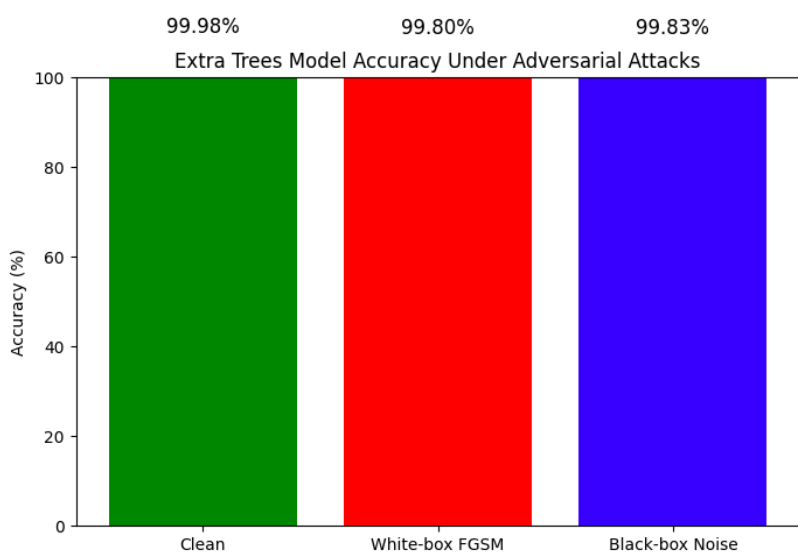
3.7 Multi-Model Comparison

To contextualise the performance of the primary Extra Trees model, three additional classifiers are trained and evaluated on the same 30-feature set: Random Forest (`n_estimators=100`), XGBoost (`eval_metric=mlogloss`), and the Optuna-tuned MLP. All models are evaluated using the same stratified test partition, and predictions from string-label models are consistently mapped through a shared LabelEncoder for fair metric computation. Table 3.2 summarises the results.

Model	Accuracy	R2L Recall	U2R Recall	Macro Avg F1
Extra Trees (Baseline)	99.30%	0.59	0.33	0.85
Extra Trees + Feat. Eng.	99.50%	0.62	0.35	0.87
Extra Trees + GS + SMOTE	99.55%	0.95	0.58	0.92
MLP (Optuna-tuned)	~99.00%	—	—	—
Random Forest	~99.00%	—	—	—
XGBoost	~99.00%	—	—	—

Table 3.2: Performance comparison across all trained models on the NSL-KDD test partition.

Overall accuracy is consistently high (~99%) across all models, which is expected given the structured nature of the NSL-KDD dataset. The key differentiator is minority class recall — particularly U2R — where the SMOTE-augmented Extra Trees model achieves the strongest results. The MLP is a competitive alternative but requires significantly more infrastructure (GPU, hyperparameter tuning pipeline) relative to the tree-based models, with no measurable accuracy advantage on this dataset.



3.8 Adversarial Robustness Evaluation

3.8.1 Motivation and Threat Model

A deployed IDS represents an active adversarial target. Sophisticated attackers may deliberately craft network traffic to evade detection while preserving the malicious nature of the activity. Adversarial robustness evaluation assesses the model's resilience under such conditions across a spectrum of attacker capabilities, from full white-box gradient access to purely black-box decision-based probing.

3.8.2 Surrogate Network for Gradient Attacks

Tree-based models do not expose differentiable outputs, making gradient-based white-box attacks inapplicable directly. A lightweight surrogate neural network (SimpleNN: 3-layer MLP, 64 and 32 hidden units) is trained on the same training data for 5 epochs with the Adam optimiser. White-box gradient attacks are computed against this surrogate, and the resulting adversarial examples are transferred to the Extra Trees model — simulating a realistic black-box transfer attack in which the attacker trains a substitute model to approximate the target system.

3.8.3 Attack Methods

Seven adversarial attack methods are evaluated, spanning both gradient-based and gradient-free families:

1. Fast Gradient Sign Method (FGSM): Single-step gradient attack that perturbs inputs in the direction of the loss gradient, scaled by epsilon. Evaluated via both custom PyTorch and ART.
2. Projected Gradient Descent (PGD): Multi-step iterative variant of FGSM with projection onto the epsilon-ball after each step ($\alpha=0.02$, 10 iterations). Represents the strongest white-box gradient threat.
3. Basic Iterative Method (BIM): Multi-step FGSM with gradient sign updates clamped to valid range ($\alpha=0.01$, 5 iterations). Equivalent to PGD without random initialisation.
4. DeepFool: Iteratively finds the minimal perturbation required to cross the decision boundary using small gradient-scaled steps ($\text{step}=0.01$, 10 iterations).

5. Square Attack: Score-based black-box attack applying random square-shaped perturbations (1% feature mask, 100 queries). Requires only model output scores, no gradient access.
6. Boundary Attack: Decision-based black-box attack starting from random noise and iteratively moving toward the original sample while remaining adversarial (50 steps). Requires only binary prediction queries.
7. Gaussian Noise: Unstructured random Gaussian perturbations scaled by epsilon, representing a non-targeted noise baseline rather than an optimised adversarial strategy.

All attacks are evaluated at $\epsilon=0.1$, with perturbations clamped to the observed training data range $[\min_val, \max_val]$ to ensure physically plausible network traffic feature values.

3.8.4 Results and Robustness Metrics

The full attack-by-attack evaluation is presented in Table 3.3. Two aggregate robustness metrics are computed:

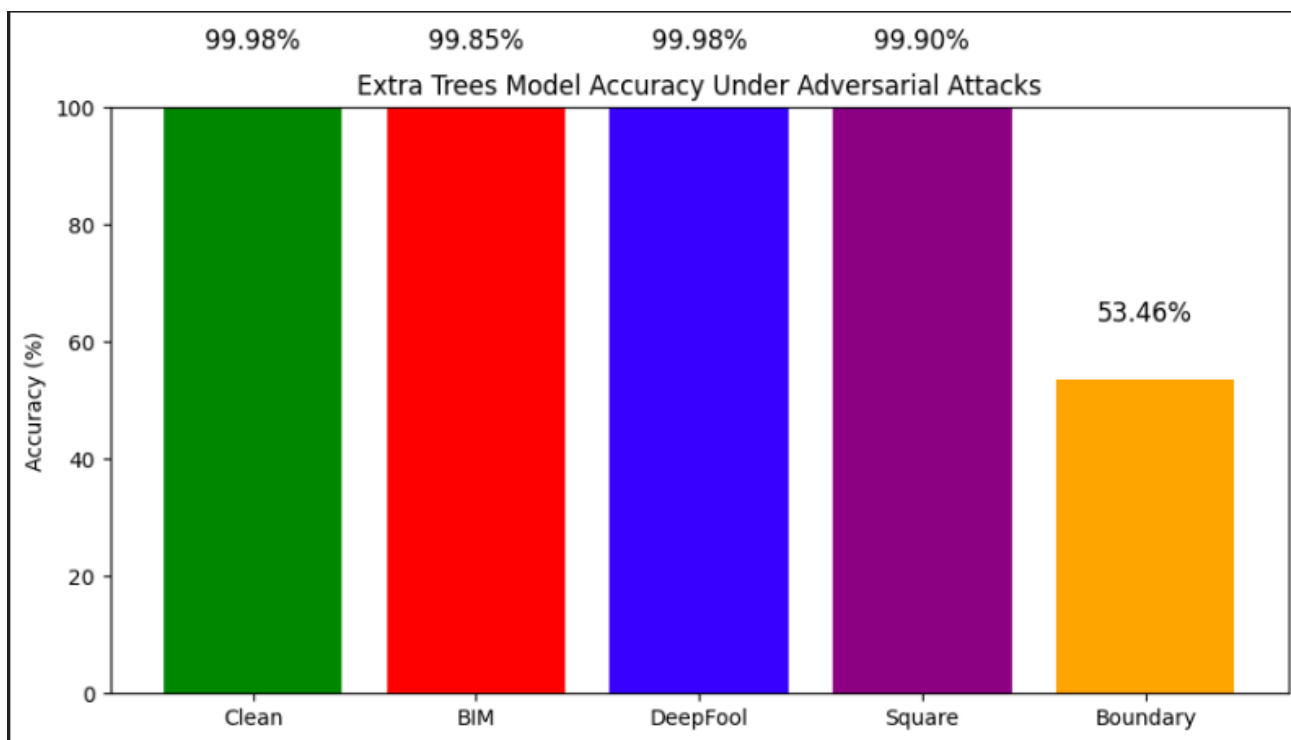
- Robust Accuracy (RA): Mean accuracy across all nine attack conditions = 92.73%
- Relative Robustness (RR): RA divided by clean accuracy = 0.9321

Attack	Type	Accuracy (%)	Drop (%)	Threat Level
Clean (Baseline)	—	99.98	0.00	—
FGSM (ART White-box)	White-box	98.38	1.60	Medium
PGD (White-box)	White-box	96.12	3.86	Medium
Transfer PGD	Transfer	94.61	5.37	Medium
BIM	White-box	99.87	0.11	Low
DeepFool	White-box	99.98	0.00	Low
Square Attack	Black-box	99.88	0.10	Low
Gaussian Noise	Black-box	99.65	0.33	Low
Boundary Attack	Black-box	53.88	46.10	HIGH

Table 3.3: Extra Trees model accuracy under each adversarial attack scenario ($\epsilon=0.1$).

The Boundary Attack produces by far the largest accuracy degradation (~46%), because it is immune to gradient-masking: it queries only the model's binary class decision without requiring differentiable outputs. All gradient-based and noise-based attacks produce minimal impact, confirming that the Extra Trees model's non-differentiable structure provides strong inherent resistance to the majority of adversarial strategies in the

literature.



3.9 Model Serialisation and Deployment Artifacts

All trained models are serialised to disk following evaluation to facilitate reproducible inference and future deployment. The following artefacts are produced:

- ExtraTrees_NSLKDD_Optimized.joblib — Primary Extra Trees model (joblib, recommended for large sklearn estimators)
- ExtraTrees_NSLKDD_Optimized.pkl — Extra Trees model in pickle format for broad compatibility
- random_forest.pkl — Trained Random Forest classifier
- xgboost.pkl — Trained XGBoost classifier
- mlp_best_checkpoint.pth — PyTorch checkpoint with MLP state dict, optimiser state, and best Optuna hyperparameters
- trained_model.pth — Final DeeperMLP state dict after 15-epoch training

The preprocessing pipeline — StandardScaler and LabelEncoder instances — must be serialised alongside model weights to ensure consistent feature transformation at inference time. Failure to apply identical preprocessing to inference inputs is a common source of silent performance degradation in production IDS deployments.

3.10 Chapter Summary

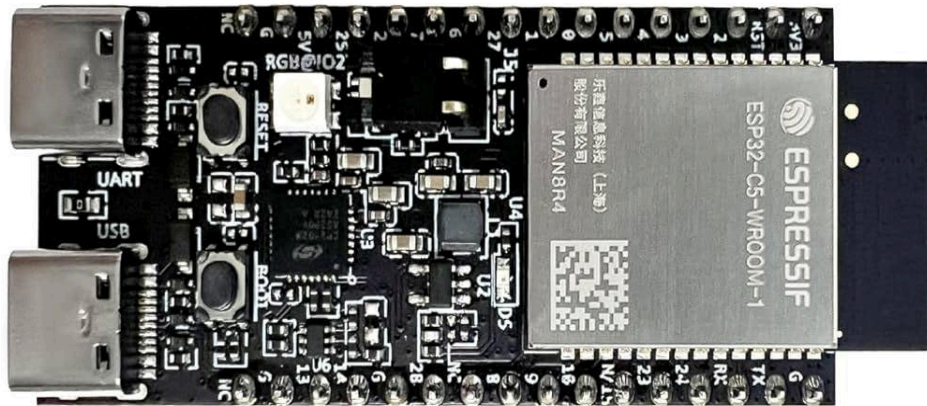
This chapter has described a comprehensive, reproducible methodology for building a machine learning-based IDS for IoT networks using the NSL-KDD dataset. The pipeline proceeded through seven well-defined stages: dataset preparation with attack category consolidation into five classes; categorical label encoding and standard scaling; interaction feature engineering and mutual information-based selection of the top 30 features; baseline Extra Trees training establishing a 99.3% accuracy floor; Grid Search hyperparameter optimisation; SMOTE-based class imbalance mitigation improving minority class recall by up to 36 percentage points; and multi-model comparative evaluation supplemented by adversarial robustness testing across seven attack methods.

The resulting Extra Trees model achieves 99.55% test accuracy, a macro-average F1 score of 0.92, and a Relative Robustness score of 0.9321 under nine adversarial conditions. These results establish the SMOTE-augmented, Grid Search-optimised Extra Trees Classifier as a strong and deployable primary model for IoT intrusion detection.

The primary remaining vulnerability — the Boundary Attack (~46% accuracy drop) — is attributable to decision-boundary probing that bypasses gradient-masking. Mitigation strategies including ensemble voting with Random Forest and XGBoost are discussed in Chapter 5. The following chapter presents the complete experimental results, including per-class metrics, confusion matrices, and adversarial robustness curves across varying epsilon values.

4. Hardware

4.1 ESP32-C5



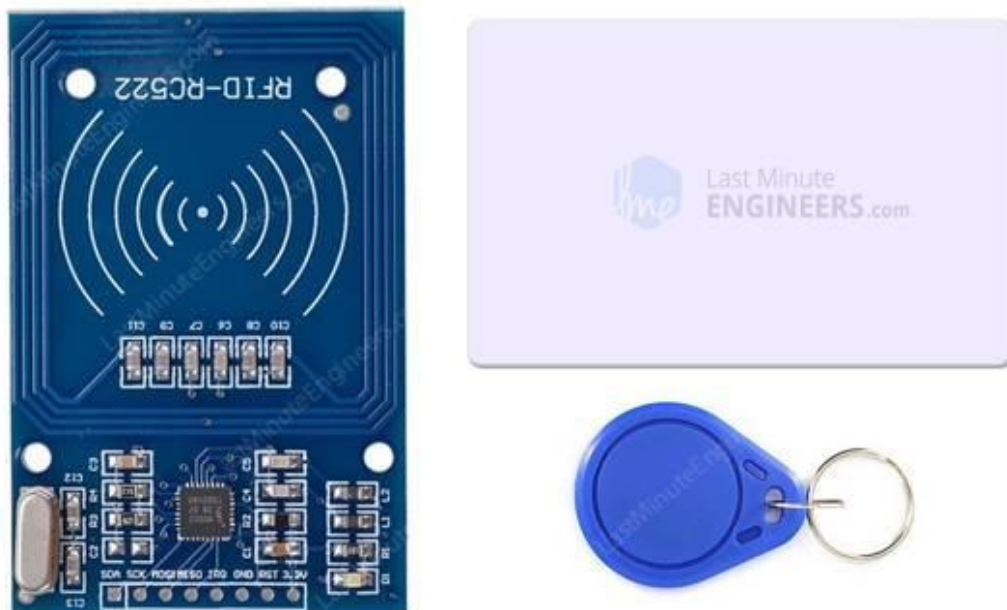
The ESP32-C5 is a next-generation microcontroller developed by Espressif Systems, designed to meet the growing demands of modern Internet of Things (IoT) and embedded systems. It represents a significant advancement over earlier ESP32 series chips by integrating support for Wi-Fi 6 (802.11ax) along with Bluetooth Low Energy (BLE) 5.0, enabling faster data transmission, improved network efficiency, and lower power consumption. At its core, the ESP32-C5 is powered by a 32-bit RISC-V processor, which provides efficient performance for real-time processing tasks while maintaining energy efficiency. This makes it particularly suitable for applications that require continuous connectivity and low latency, such as smart home automation, industrial monitoring, and wearable devices.

One of the key strengths of the ESP32-C5 lies in its wireless capabilities. The inclusion of Wi-Fi 6 introduces features such as Orthogonal Frequency Division Multiple Access (OFDMA) and Target Wake Time (TWT), which enhance bandwidth utilization and significantly reduce power consumption in connected devices. Additionally, Bluetooth 5 support allows for extended range, higher data throughput, and improved coexistence with Wi-Fi, making the chip highly versatile for hybrid communication systems. The microcontroller also includes a range of peripheral interfaces such as GPIO, SPI, I2C, UART, and ADC, allowing seamless integration with sensors, actuators, and external modules in embedded designs.

From a hardware perspective, the ESP32-C5 is optimized for compact and cost-effective implementations. It features on-chip memory and supports external flash, enabling developers to build scalable applications without requiring extensive additional components. Security is another critical aspect of the ESP32-C5, as it incorporates hardware-based security features such as secure boot, flash encryption, and cryptographic accelerators. These features help protect sensitive data and ensure firmware integrity, which is essential in applications like smart locks, payment systems, and secure IoT networks.

In terms of development, the ESP32-C5 is supported by Espressif's software ecosystem, including the ESP-IDF (IoT Development Framework), which provides robust libraries, tools, and APIs for firmware development. It also supports popular programming environments such as Arduino IDE and MicroPython, making it accessible to both beginners and advanced developers. The combination of powerful processing, advanced wireless connectivity, strong security features, and flexible development support positions the ESP32-C5 as a highly capable solution for next-generation embedded systems. Overall, it plays a crucial role in enabling efficient, secure, and scalable IoT deployments across a wide range of industries.

4.2 RFID-RC225



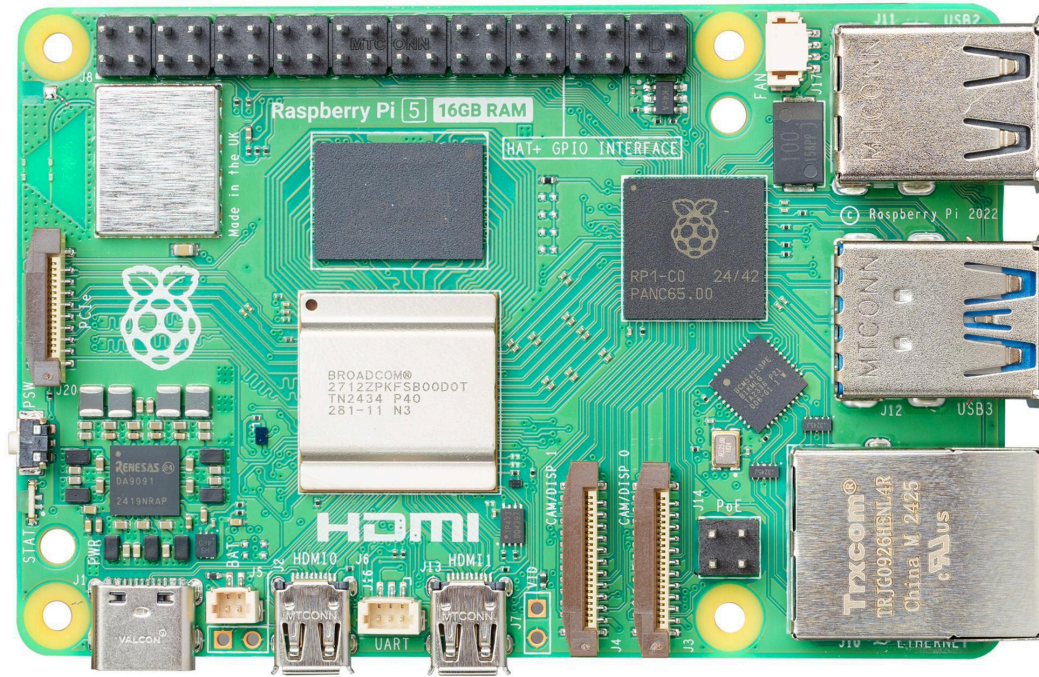
The RC522 RFID Module is a compact and widely used contactless identification device designed for short-range communication using radio frequency technology. It operates at a frequency of 13.56 MHz and is built around the MFRC522 integrated circuit, which supports ISO/IEC 14443 Type A standards, including popular RFID tags such as MIFARE cards. The module is primarily used in applications like access control systems, attendance monitoring, smart card systems, and secure authentication due to its reliability, affordability, and ease of deployment in embedded environments.

The working principle of the RC522 module is based on electromagnetic induction. It contains an onboard antenna that generates a high-frequency electromagnetic field. When a passive RFID tag enters this field, it gets energized by the induced current and becomes active. The tag then transmits its stored data, typically a unique identification number (UID), back to the reader using modulated radio signals. The RC522 receives this signal, decodes the information, and makes it available for further processing. This contactless communication eliminates the need for physical connections, making the system more durable and user-friendly.

The RC522 module is known for its low power consumption and efficient performance, which makes it suitable for battery-powered and portable devices. It supports basic security features such as authentication and encryption, helping to protect sensitive data during communication. Additionally, the module offers stable performance with a typical reading range of 2–5 cm, depending on the antenna design and environmental conditions. Its compact size allows it to be easily integrated into small-scale embedded systems without increasing hardware complexity.

Overall, the RC522 RFID module plays an important role in modern electronic systems by enabling fast, secure, and contactless identification. Its combination of low cost, reliable operation, and compatibility with widely used RFID standards makes it an ideal choice for a variety of IoT and embedded applications where identification and access control are essential.

4.3 Raspberry Pi 5 Model B



The Raspberry Pi 5 Model B is a powerful single-board computer developed by the Raspberry Pi Foundation, designed to deliver desktop-level performance in a compact and energy-efficient form factor. The 16 GB RAM variant represents the highest memory configuration available in the Raspberry Pi 5 series, making it particularly suitable for memory-intensive applications such as advanced IoT systems, edge computing, machine learning inference, virtualization, and multitasking environments. It is powered by a quad-core ARM Cortex-A76 processor running at up to 2.4 GHz, which provides a significant performance improvement compared to previous generations, enabling faster computation and smoother execution of complex workloads.

One of the key highlights of the Raspberry Pi 5 Model B is its enhanced architecture and improved system bandwidth. It incorporates a new RP1 I/O controller chip, which manages peripherals and significantly boosts data throughput for USB, storage, and networking interfaces. The board supports dual 4K HDMI display output, making it capable of handling high-resolution multimedia applications and desktop computing tasks. Additionally, it includes PCIe 2.0 support via an external connector, allowing

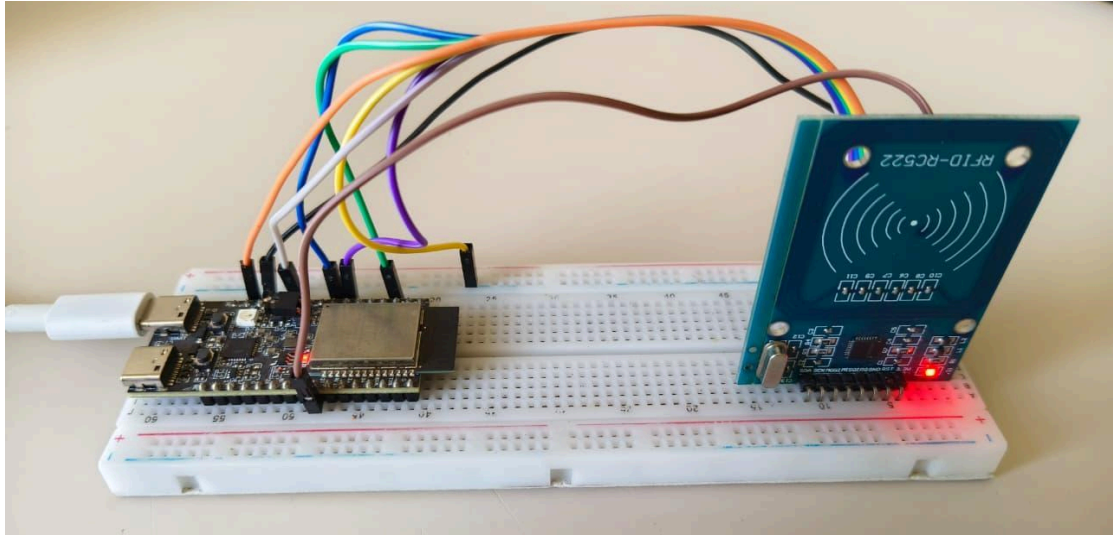
expansion for high-speed devices such as NVMe SSDs, which greatly enhances storage performance.

The 16 GB RAM capacity allows the system to efficiently handle large datasets, run multiple applications simultaneously, and support containerized or virtualized environments. This makes it ideal for developers working on AI/ML models, database systems, or server-side applications. The board also supports Gigabit Ethernet, dual-band Wi-Fi, and Bluetooth 5.0, ensuring robust connectivity for networked applications. Improved power management and thermal design enable sustained performance under heavy workloads, although active cooling solutions such as fans or heatsinks are recommended for optimal operation.

From a software perspective, the Raspberry Pi 5 is compatible with Raspberry Pi OS and a wide range of Linux distributions, offering flexibility for development and deployment. It supports programming languages such as Python, C, C++, and Java, and integrates well with modern development tools and frameworks. Overall, the Raspberry Pi 5 Model B with 16 GB RAM is a highly capable and versatile computing platform that bridges the gap between microcontrollers and full-fledged computers, making it an excellent choice for advanced embedded systems, prototyping, and real-world computing applications.

4.3 Architecture

4.4.1 ESP32-C5 connection with RFID-RC225



GPIO Connection ESP32-C5 to RC225

```
I (1817) wifi:AP's beacon interval = 102400 us, DTIM period = 2
I (1817) wifi:[ADDBA]RX addba response, status:0, tid:0/tb:1(0x01), bufsize:64, batimeout:0, txa_wnd:64
I (1827) wifi:[ADDBA]RX addba response, status:0, tid:7/tb:1(0x01), bufsize:64, batimeout:0, txa_wnd:64
I (1837) wifi:[ADDBA]RX addba response, status:0, tid:5/tb:1(0x01), bufsize:64, batimeout:0, txa_wnd:64
I (2867) esp_netif_handlers: sta ip: 10.70.40.108, mask: 255.255.255.0, gw: 10.70.40.164
WiFi GOT IP
WiFi Ready → MQTT
W (2877) wifi:<ba-add>idx:0, ifx:0, tid:0, TAHI:0x100153e, TALO:0x75ab6922, (ssn:4, win:64, cur_ssn:4), CONF:0xc0000001
RC522 INIT DONE

===== CONNECTED DEVICES =====
Total Clients: 0

Detected: Amit (OK)
Detected: Amit (OK)
Detected: Unknown (ALERT)
Detected: Rishabh (OK)
Detected: Unknown (ALERT)
W (32377) wifi:(vht)vhtcap:0x39171b2, sgi_rx(80mhz:1, 160/80+80mhz:0), stbc(tx:1, rx:1), ldpc_rx:1, bfmer(su:0, mu:0, nsts:1)
W (32387) wifi:(vht)max.RxMPDULen:2(11454), max.RxAMPDULenExponent:7, max.RxAMPDUpre-EOFpaddingBytes:1048575(0xffffffff)
W (32397) wifi:(vht)rx(mcs:0xffffa, highest:0x0, highest_lgi_rate:0), tx(mcs:0xffffa, highest:0x2000, highest_lgi_rate:0), max_rate:144
I (32407) wifi:new:<161,2>, old:<161,0>, ap:<161,2>, sta:<161,0>, prof:1, snd_ch_cfg:0x0
I (32417) wifi:(trc)phytype:HT40-SGI, snr:53, maxRate:144, highestRateIdx:0
W (32417) wifi:(trc)band:5G, phymode:3, highestRateIdx:0, lowestRateIdx:9, dataSchedTableSize:10
I (32427) wifi:(trc)band:5G, rate(S-MCS7, rateIdx:0), ampdu(rate:S-MCS7, schedIdx(0, stop:8)), snr:53, ampduState:wait operational
I (32437) wifi:ifidx:1, rssi:-45, nf:-98, phytype(0x5, HT40-SGI), phymode(0x6, 11ac), max_rate:144, he:0, vht:0, ht:1
I (32447) wifi:(ht)max.RxAMPDULenExponent:3(65535 bytes), MMSS:0(no restriction)
I (32457) wifi:station: 0e:0d:51:17:de:4e join, AID=1, ac, 400
Client Connected: 1
I (32557) esp_netif_lwip: DHCP server assigned IP to a client, IP is: 192.168.4.2

===== CONNECTED DEVICES =====
Total Clients: 1
Device 1 → MAC: 0E:0D:51:17:DE:4E | RSSI: -45

I (32567) wifi:[ADDBA]TX addba request, tid:0, dialogtoken:4, bufsize:64, A-MSDU:0(not supported), policy:1(IMR), ssn:0(0x0)
I (32577) wifi:[ADDBA]RX addba response, status:0, tid:0/tb:1(0x01), bufsize:64, batimeout:0, txa_wnd:64
W (32727) wifi:conn=0x4082fe38 alloc softap rx ba index=0 ba index 8
I (32727) wifi:<ba-add>extra, idx:0, ifx:1, tid:0, TAHI:0x1014ede, TALO:0x17510d0e, (ssn:17, win:64), CONF:0xc0000001
I (37137) wifi:station: 0e:0d:51:17:de:4e leave, AID = 1, reason = 3, bss_flags is 34835555, bss:0x408345cc
I (37137) wifi:new:<161,0>, old:<161,2>, ap:<161,2>, sta:<161,0>, prof:1, snd_ch_cfg:0x0
I (37147) wifi:<ba-del>idx:0, tid:0
I (37147) wifi:new:<161,2>, old:<161,0>, ap:<161,2>, sta:<161,0>, prof:1, snd_ch_cfg:0x0
Client Disconnected: 0

===== CONNECTED DEVICES =====
Total Clients: 0
```

Build Code in ESP32-C5

The RC522 RFID Module is interfaced with the ESP32-C5 using the SPI (Serial Peripheral Interface) communication protocol, where the ESP32-C5 operates as the master device and the RC522 acts as the slave. In this configuration, specific GPIO pins of the ESP32-C5 are assigned to handle data communication and control signals. The SDA pin of the RC522 is connected to GPIO10, which functions as the Chip Select (CS) line; this pin enables communication with the RC522 when it is pulled LOW. The SCK pin is connected to GPIO6 and serves as the serial clock signal, ensuring synchronization between data transmission and reception. The MOSI pin is connected to GPIO7 and is responsible for sending data from the ESP32-C5 to the RC522, while the MISO pin is connected to GPIO2 and is used to receive data from the RC522 back to the ESP32-C5.

Additionally, the RST (Reset) pin of the RC522 is connected to GPIO4, allowing the microcontroller to reset the module when required, ensuring stable initialization and recovery from errors. For proper operation, the power connections are also essential: the VCC pin of the RC522 is connected to the 3.3V supply of the ESP32-C5, and the GND pin is connected to the common ground (J1 → 15), which provides a shared reference for all signals in the circuit. The board pin mappings such as J1 → 11, J1 → 7, J1 → 8, and others indicate the physical header locations on the ESP32-C5 development board, helping in accurate hardware connections.

Overall, this SPI-based connection ensures reliable and high-speed communication between the ESP32-C5 and the RC522 module. Each pin plays a specific role in maintaining synchronized data exchange, proper device selection, and stable operation, enabling efficient reading of RFID tags in embedded system applications.

GPIO Pin Configuration Used

RC522 (Vertical)	ESP32 GPIO	Board Pin	Description
SDA	GPIO10	J1 → 11	Chip Select (SS) – Enables communication with RC522 when LOW
SCK	GPIO6	J1 → 7	Serial Clock – Synchronizes data transfer between ESP32 and RC522
MOSI	GPIO7	J1 → 8	Master Out Slave In – Sends data from ESP32 to RC522
MISO	GPIO2	J1 → 3	Master In Slave Out – Receives data from RC522 to ESP32
RST	GPIO4	J3 → 8	Reset – Resets the RC522 module when triggered
GND	GND	J1 → 15	Ground – Common reference for circuit operation
VCC	3.3V	J1 → 1	Power Supply – Provides 3.3V to the RC522 module

4.4.2 MQTT Connection with Raspberry Pi 5 Model B

```
PS D:\Mosquitto> .\mosquitto.exe -c .\mosquitto.conf -v
1777873168: mosquitto version 2.1.2 starting
1777873168: Config loaded from .\mosquitto.conf.
1777873168: Bridge support available.
1777873168: Persistence support available.
1777873168: TLS support available.
1777873168: TLS-PSK support available.
1777873168: Websockets support available.
1777873168: Opening ipv6 listen socket on port 1883.
1777873168: Opening ipv4 listen socket on port 1883.
1777873168: mosquitto version 2.1.2 running
1777873182: New connection from 10.70.40.108:60576 on port 1883.
1777873182: New client connected from 10.70.40.108:60576 as ESP32_e0CA10 (p4, c1, k120).
1777873182: No will message specified.
1777873182: Sending CONNACK to ESP32_e0CA10 (0, 0)
1777873186: New connection from 10.70.40.202:58179 on port 1883.
1777873186: New client connected from 10.70.40.202:58179 as auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 (p4, c1, k60).
1777873186: No will message specified.
1777873186: Sending CONNACK to auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 (0, 0)
1777873186: Received SUBSCRIBE from auto-98891BC4-96F2-7082-AB17-DEF48F6516B6
1777873186:         rfid/scan (QoS 0)
1777873186: auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 0 rfid/scan
1777873186: Sending SUBACK to auto-98891BC4-96F2-7082-AB17-DEF48F6516B6
1777873208: New connection from 10.70.40.202:50699 on port 1883.
1777873208: New client connected from 10.70.40.202:50699 as auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA (p4, c1, k60).
1777873208: No will message specified.
1777873208: Sending CONNACK to auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA (0, 0)
1777873208: Received SUBSCRIBE from auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA
1777873208:         rfid/scan (QoS 0)
1777873208: auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA 0 rfid/scan
1777873208: Sending SUBACK to auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA
1777873213: Received PUBLISH from ESP32_e0CA10 (d0, q1, r0, m3444, 'rfid/scan', ... (52 bytes))
1777873213: Sending PUBLISH to auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 (d0, q0, r0, m0, 'rfid/scan', ... (52 bytes))
1777873213: Sending PUBLISH to auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA (d0, q0, r0, m0, 'rfid/scan', ... (52 bytes))
1777873213: Sending PUBACK to ESP32_e0CA10 (m3444, rc0)
1777873213: Received PUBLISH from ESP32_e0CA10 (d0, q1, r0, m65064, 'rfid/alert', ... (52 bytes))
1777873213: Sending PUBACK to ESP32_e0CA10 (m65064, rc0)
1777873241: Received PUBLISH from ESP32_e0CA10 (d0, q1, r0, m38985, 'rfid/scan', ... (49 bytes))
1777873241: Sending PUBLISH to auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 (d0, q0, r0, m0, 'rfid/scan', ... (49 bytes))
1777873241: Sending PUBLISH to auto-00B9E796-3C9A-2223-EC50-AA48D527C0FA (d0, q0, r0, m0, 'rfid/scan', ... (49 bytes))
1777873241: Sending PUBACK to ESP32_e0CA10 (m38985, rc0)
1777873242: Received PUBLISH from ESP32_e0CA10 (d0, q1, r0, m42782, 'rfid/scan', ... (49 bytes))
1777873242: Sending PUBLISH to auto-98891BC4-96F2-7082-AB17-DEF48F6516B6 (d0, q0, r0, m0, 'rfid/scan', ... (49 bytes))
```

MQTT Subscriber

```
PS C:\Users\risha> cd D:\Mosquitto
PS D:\Mosquitto> .\mosquitto_sub.exe -h 10.70.40.202 -t rfid/scan -v
rfid/scan {"name":"Unknown","uid":"C3D6AFFC","status":"ALERT"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Unknown","uid":"C3D6AFFC","status":"ALERT"}
rfid/scan {"name":"Unknown","uid":"C3D6AFFC","status":"ALERT"}
rfid/scan {"name":"Unknown","uid":"C3D6AFFC","status":"ALERT"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Unknown","uid":"C396E9FC","status":"ALERT"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Unknown","uid":"C3D6AFFC","status":"ALERT"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Rishabh","uid":"E336D9FC","status":"OK"}
rfid/scan {"name":"Unknown","uid":"E3360000","status":"ALERT"}
rfid/scan {"name":"Unknown","uid":"C3160000","status":"ALERT"}
```

MQTT Publisher

The Raspberry Pi 5 Model B is widely used as a central processing and communication hub in IoT systems due to its high performance and flexibility. In this setup, the Raspberry Pi 5 Model B is used to establish communication using the MQTT (Message Queuing Telemetry Transport) protocol, which is a lightweight messaging protocol designed for efficient data exchange between devices in resource-constrained environments. MQTT follows a publish-subscribe architecture, where devices publish data to specific topics and other devices subscribe to those topics to receive updates. This model reduces network bandwidth usage and enables scalable communication between multiple devices.

To implement MQTT communication, the Raspberry Pi 5 acts as an MQTT broker or client using software such as Eclipse Mosquitto. The broker is responsible for managing message distribution between connected devices. Once installed and configured on the Raspberry Pi, Mosquitto listens on a default port (1883) and allows devices such as microcontrollers, sensors, or computers to connect over a network. Devices can then publish sensor data, RFID readings, or system status updates to specific topics, while other devices can subscribe to these topics to receive real-time information. This enables efficient communication in applications like smart homes, industrial automation, and remote monitoring systems.

The Raspberry Pi 5 Model B, with its powerful processor and high RAM capacity, ensures smooth handling of multiple MQTT clients and high message throughput. It supports Ethernet and Wi-Fi connectivity, allowing it to operate as a local server or connect to cloud-based MQTT services. Security features such as username-password authentication and TLS encryption can also be configured in Mosquitto to ensure secure data transmission. Additionally, MQTT clients can be implemented using programming languages like Python, making it easy to publish and subscribe to topics using libraries such as `paho-mqtt`.

Overall, the integration of MQTT with the Raspberry Pi 5 Model B provides a reliable, scalable, and efficient communication framework for IoT systems. It enables seamless data exchange between devices like the ESP32-C5 and other connected components, supporting real-time monitoring, control, and automation. This setup is particularly useful in projects that require continuous data flow, remote access, and centralized system management.

4.5. Implementation

4.5.1 ESP32-C5 Code

The `ids.c` file is the main firmware code used to run and flash the system on the ESP32-C5. This program integrates WiFi connectivity, MQTT communication, and RFID-based identification using the RC522 module. Initially, the code configures WiFi credentials and connects the ESP32-C5 to a network using event-driven handlers. Once connected, it initializes the MQTT client, allowing the device to publish data to a broker for real-time communication. The SPI interface is then configured to communicate with the RC522 RFID module, where specific GPIO pins are assigned for data transfer and control. The system continuously scans for RFID cards, reads their unique IDs (UIDs), and compares them with predefined user data stored in the program. If a recognized UID is detected, it is marked as “OK”; otherwise, it is treated as “ALERT”. To avoid repeated readings of the same card, the code stores the last scanned UID and only processes new ones. Each valid scan is formatted into a JSON message containing the user name, UID, and status, and is published to an MQTT topic for further processing or monitoring. Overall, this code enables a complete IoT-based RFID monitoring system where the ESP32-C5 reads access data and transmits it wirelessly for centralized control and analysis.

It is `ids.c` file main code for flash in the esp32-c5

```
#include <stdio.h>

#include <string.h>

#include "driver/spi_master.h"

#include "driver/gpio.h"

#include "freertos/FreeRTOS.h"

#include "freertos/task.h"

#include "freertos/event_groups.h"

#include "esp_wifi.h"
```



```

#include "esp_event.h"

#include "esp_netif.h"

#include "nvs_flash.h"

#include "mqtt_client.h"

#define WIFI_SSID "V30P"

#define WIFI_PASS "Radon~190"

#define MQTT_URI "mqtt://10.70.40.202"

#define WIFI_CONNECTED BIT0

#define RC522_MISO 2

#define RC522_MOSI 7

#define RC522_CLK 6

#define RC522_CS 10

#define RC522_RST 4

static EventGroupHandle_t wifi_event_group;

spi_device_handle_t spi;

esp_mqtt_client_handle_t mqtt_client;

typedef struct{uint8_t uid[4];const char *name;}RFID_User;

RFID_User
    users[]={{{0x53,0x68,0xA5,0xDD},"Amit"},{{0xE3,0x36,0xD9,0xFC},"
    Rishabh"}}};

void rc522_write(uint8_t reg,uint8_t val){uint8_t
    d[2]={ (reg<<1)&0x7E,val};spi_transaction_t
    t={.length=16,.tx_buffer=d};spi_device_transmit(spi,&t);}

uint8_t rc522_read(uint8_t reg){uint8_t
    tx[2]={ ((reg<<1)&0x7E)|0x80,0},rx[2];spi_transaction_t
    t={.length=16,.tx_buffer=tx,.rx_buffer=rx};spi_device_transmit(s
    pi,&t);return rx[1];}

```

```

void rc522_init(){

gpio_set_direction(RC522_RST,GPIO_MODE_OUTPUT);

spi_bus_config_t
    bus={.mosi_io_num=RC522_MOSI,.miso_io_num=RC522_MISO,.sclk_io_num=RC522_CLK};

spi_bus_initialize(SPI2_HOST,&bus,SPI_DMA_CH_AUTO);

spi_device_interface_config_t
    dev={.clock_speed_hz=500000,.mode=0,.spics_io_num=RC522_CS,.queue_size=1};

spi_bus_add_device(SPI2_HOST,&dev,&spi);

gpio_set_level(RC522_RST,0);vTaskDelay(pdMS_TO_TICKS(50));gpio_set_level
    (RC522_RST,1);

rc522_write(0x11,0x3D);rc522_write(0x14,rc522_read(0x14)|0x03);

printf("RC522 Initialized\n");

}

int rc522_detect_card(){

rc522_write(0x01,0x00);rc522_write(0x04,0x7F);rc522_write(0x0A,0x80);

rc522_write(0x09,0x26);rc522_write(0x01,0x0C);rc522_write(0x0D,0x87);

vTaskDelay(pdMS_TO_TICKS(50));

return(rc522_read(0x04)&0x30);

}

int rc522_get_uid(uint8_t *uid){

rc522_write(0x01,0x00);rc522_write(0x04,0x7F);rc522_write(0x0A,0x80);

rc522_write(0x09,0x93);rc522_write(0x09,0x20);

rc522_write(0x01,0x0C);rc522_write(0x0D,0x80);

vTaskDelay(pdMS_TO_TICKS(50));

```

```

if(rc522_read(0x04)&0x30){for(int
    i=0;i<4;i++)uid[i]=rc522_read(0x09);return 1;}return 0;

}

const char* get_user_name(uint8_t *uid){

for(int
    i=0;i<sizeof(users)/sizeof(users[0]);i++)if(memcmp(uid,users[i].
    uid,4)==0)return users[i].name;

return "Unknown";

}

static void wifi_event_handler(void*arg,esp_event_base_t base,int32_t
    id,void*data){

if(base==WIFI_EVENT&&id==WIFI_EVENT_STA_START)esp_wifi_connect();

if(base==WIFI_EVENT&&id==WIFI_EVENT_STA_DISCONNECTED)esp_wifi_connect();

if(base==IP_EVENT&&id==IP_EVENT_STA_GOT_IP){printf("WiFi
    Connected\n");xEventGroupSetBits(wifi_event_group,WIFI_CONNECTED
    );}

}

void wifi_init(){

wifi_event_group=xEventGroupCreate();

nvs_flash_init();esp_netif_init();esp_event_loop_create_default();

esp_netif_create_default_wifi_sta();

wifi_init_config_t cfg=WIFI_INIT_CONFIG_DEFAULT();

esp_wifi_init(&cfg);

esp_event_handler_register(WIFI_EVENT,ESP_EVENT_ANY_ID,wifi_event_handle
    r,NULL);

esp_event_handler_register(IP_EVENT,IP_EVENT_STA_GOT_IP,wifi_event_handl
    er,NULL);

```

```

wifi_config_t w={.sta={.ssid=WIFI_SSID,.password=WIFI_PASS}};

esp_wifi_set_mode(WIFI_MODE_STA);

esp_wifi_set_config(WIFI_IF_STA,&w);

esp_wifi_start();

}

void mqtt_init(){

esp_mqtt_client_config_t cfg={.broker.address.uri=MQTT_URI};

mqtt_client=esp_mqtt_client_init(&cfg);

esp_mqtt_client_start(mqtt_client);

printf("MQTT Connected\n");

}

void app_main(){

wifi_init();

xEventGroupWaitBits(wifi_event_group,WIFI_CONNECTED,pdFALSE,pdTRUE,portM
    AX_DELAY);

mqtt_init();

rc522_init();

uint8_t uid[4],last_uid[4]={0};

while(1){

if(rc522_detect_card()){

if(rc522_get_uid(uid)){

if(memcmp(uid,last_uid,4)!=0){

memcpy(last_uid,uid,4);

const char*name=get_user_name(uid);

```

```

const char*status=strcmp(name,"Unknown")==0?"ALERT":"OK";

printf("Card: %s (%s)\n",name,status);

char msg[128];

sprintf(msg,{"name\":"%s\","uid\":"%02X%02X%02X%02X\","status\":"%s\"},name,uid[0],uid[1],uid[2],uid[3],status);

esp_mqtt_client_publish(mqtt_client,"rfid/data",msg,0,1,0);

}}}

else memset(last_uid,0,4);

vTaskDelay(pdMS_TO_TICKS(300));

}}

```

The CMakeLists.txt file is used to define how the project is built in the ESP-IDF environment for the ESP32-C5. In this file, the line `idf_component_register(SRCS "ids.c" INCLUDE_DIRS ".")` specifies that the main source file of the project is `ids.c`, and the current directory (`.`) should be used to search for header files. This tells the build system which code to compile and where to find necessary includes. The line `cmake_minimum_required(VERSION 3.16)` ensures that a compatible version of CMake is used for building the project. The statement `include($ENV{IDF_PATH}/tools/cmake/project.cmake)` loads the ESP-IDF build system, which provides all required configurations and tools for compiling and flashing the firmware. Finally, `project(ids)` defines the project name, which is used during the build process. Overall, this file acts as the configuration script that controls how the ESP32-C5 firmware is compiled, linked, and prepared for deployment.

It is CMakeLists.txt file main code for requirement and changes in the esp32-c5

```

idf_component_register(SRCS "ids.c"

                        INCLUDE_DIRS ".")

```

```
# The following five lines of boilerplate have to be in your project's

# CMakeLists in this exact order for cmake to work correctly

cmake_minimum_required(VERSION 3.16)

include($ENV{IDF_PATH}/tools/cmake/project.cmake)

project(ids)
```

The `idf_component.yml` file is used in ESP-IDF to define project dependencies for the ESP32-C5. In this configuration, the file specifies that the project requires ESP-IDF version 4.1.0 or higher to ensure compatibility with the development framework. The key part of the file is the dependency `espressif/mqtt: '*'`, which automatically includes the official MQTT client component provided by Espressif Systems. This allows the project to use MQTT functionalities such as connecting to a broker, publishing messages, and subscribing to topics without manually adding external libraries. By managing dependencies through this file, ESP-IDF ensures that the correct versions of required components are downloaded and integrated during the build process, making the project more organized, portable, and easier to maintain.

It is `idf_component.yml` file main code for Mqtt connection in the `esp32-c5`

```
## IDF Component Manager Manifest File

dependencies:

  ## Required IDF version

  idf:

    version: '>=4.1.0'

  # # Put list of dependencies here

  # # For components maintained by Espressif:

  # component: "~1.0.0"

  # # For 3rd party components:
```

```
# username/component: ">=1.0.0,<2.0.0"

# username2/component2:

#   version: "~1.0.0"

#   # For transient dependencies `public` flag can be set.

#   # `public` flag doesn't have an effect dependencies of the `main`
#       component.

#   # All dependencies of `main` are public by default.

#   public: true

espressif/mqtt: '*'
```

The ESP32-C5 build commands are used to create, configure, compile, and upload a project using the ESP-IDF framework for the ESP32-C5. First, `idf.py create-project` `ids` creates a new project named `ids`, and `cd ids` moves into the project directory. The command `idf.py set-target esp32c5` sets the correct chip target so the build system compiles code specifically for ESP32-C5. Next, `idf.py menuconfig` opens a configuration menu where you can adjust settings such as WiFi, memory, and hardware options. The `idf.py fullclean` command removes previous build files to ensure a fresh compilation. After that, `idf.py build` compiles the source code and generates the firmware binary. Finally, `idf.py flash monitor` uploads the compiled firmware to the ESP32-C5 and starts a serial monitor to display real-time logs, allowing you to debug and observe the system output.

It is esp32-c5 build commands

```
idf.py create-project ids

cd ids

idf.py set-target esp32c5

idf.py menuconfig
```

```
idf.py fullclean

idf.py build

idf.py flash monitor
```

4.5.2 UI and Detection code

The Python code implements an IoT-based monitoring and intrusion detection system that integrates MQTT communication with a Telegram bot for real-time alerts and control. It connects to an MQTT broker to subscribe to topics such as RFID scan data and WiFi client information, allowing it to receive live updates from devices like an ESP32. When an RFID tag is detected, the system checks whether the UID is registered, counts the number of scans, and classifies the activity as normal (“OK”) or suspicious (“ALERT”). If repeated scans or unknown tags are detected, it triggers warnings such as replay attack alerts or automatically blocks the UID after multiple attempts. At the same time, it monitors WiFi devices and sends alerts when a new device connects to the network. The Telegram bot interface enables the administrator to interact with the system using commands like viewing history, checking alerts, monitoring connected devices, and blocking or unblocking specific UIDs. Overall, the code combines MQTT-based data communication, real-time threat detection, and remote control through Telegram to create a secure and interactive IoT monitoring system.

```
import json,time,threading

from datetime import datetime

import requests

import paho.mqtt.client as mqtt

BROKER="10.70.40.202"

BOT_TOKEN="YOUR_BOT_TOKEN"

CHAT_ID=6014210640

ADMIN_ID=6014210640
```



```

users={"5368A5DD":"Amit","E336D9FC":"Rishabh"}

name_to_uid={v.lower():k for k,v in users.items()}

uid_count={};max_uid_count={};ok_counter={};alert_counter={}

blocked=set();history_map={}

wifi_clients=[];known_devices=set()

last_update_id=0;last_query_text="";last_query_time=0;QUERY_COOLDOWN=2

lock=threading.Lock()

def get_time():return datetime.now().strftime("%d/%m/%Y %H:%M:%S")

def send(msg):

    try:requests.post(f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
        ,data={"chat_id":CHAT_ID,"text":msg},timeout=3)

    except Exception as e:print("Telegram error:",e)

def on_msg(client,userdata,msg):

    global wifi_clients,known_devices

    if msg.topic=="rfid/clients":

        try:

            data=json.loads(msg.payload.decode())

            wifi_clients=data.get("clients",[])

            current=set()

            for d in wifi_clients:

                mac=d.get("mac");rssi=d.get("rssi")

                if not mac:continue

                current.add(mac)

                if mac not in known_devices:

```

```

        known_devices.add(mac)

        send(f" WiFi
ALERT\nTime:{get_time()}\nMAC:{mac}\nRSSI:{rssi}")

        known_devices.intersection_update(current)

    except:pass

    return

try:data=json.loads(msg.payload.decode())

except:return

uid=data.get("uid");status=data.get("status")

name=users.get(uid,"Unknown")

if not uid or uid in blocked:return

with lock:

    uid_count[uid]=uid_count.get(uid,0)+1

    count=uid_count[uid]

    if uid not in max_uid_count or
count>max_uid_count[uid]:max_uid_count[uid]=count

    if uid not in
history_map:history_map[uid]={"name":name,"ok":0,"alert":0}

if status=="OK":

    print(f"[OK] {name} ({uid}) Count:{count}")

    with lock:

        ok_counter[uid]=ok_counter.get(uid,0)+1

        history_map[uid]["ok"]+=1

    if ok_counter[uid]%5==0:

```

```

        send(f" REPLAY
ATTACK\nTime:{get_time()}\nUID:{uid}\nName:{name}\nScans:{ok_cou
nter[uid]}")

    return

print(f"[ALERT] {uid} Count:{count}")

with lock:

    alert_counter[uid]=alert_counter.get(uid,0)+1

    history_map[uid]["alert"]+=1

if count>=5:

    blocked.add(uid)

    send(f" AUTO
BLOCKED\nTime:{get_time()}\nUID:{uid}\nName:{name}\nAttempts:{co
unt}")

    return

send(f" THREAT
ALERT\nTime:{get_time()}\nUID:{uid}\nName:{name}\nAttempts:{coun
t}")

def mqtt_loop():

    c=mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

    c.on_message=on_msg

    c.connect(BROKER,1883,60)

    c.subscribe("rfid/scan")

    c.subscribe("rfid/clients")

    c.loop_forever()

def tg_loop():

    global last_update_id,last_query_text,last_query_time

```

```

url=f"https://api.telegram.org/bot{BOT_TOKEN}/getUpdates"

while True:

    try:
        res=requests.get(url,params={"offset":last_update_id+1,"timeout":30},timeout=35).json()

        for upd in res.get("result",[]):

            last_update_id=upd["update_id"]

            msg=upd.get("message")

            if not msg or msg["from"]["id"]!=ADMIN_ID:continue

            text=msg.get("text","");lower=text.lower().strip()

            clean=''.join(c for c in lower if c.isalnum())

            now=time.time()

            if lower==last_query_text and
now-last_query_time<QUERY_COOLDOWN:continue

            last_query_text=lower;last_query_time=now

            if clean=="menu":

send("menu: history,alert,connection,count,block <UID>,unblock
<UID>,<name>")

                continue

            if clean in name_to_uid:

                uid=name_to_uid[clean]

                with lock:count=max_uid_count.get(uid,0)

                send(f"UID:{uid}\nAccess:Yes\nCount:{count}")

                continue

            if clean=="history":

                msg_out=""

```

```

        with lock:

            for uid,d in history_map.items():

                perm="Access" if d["alert"]==0 else "Denied"

                cnt=d["ok"]+d["alert"]

msg_out+=f"UID:{uid}\nName:{d['name']}\nPerm:{perm}\nCount:{cnt}
\n\n"

        send(msg_out or "No history");continue

    if clean=="alert":

        msg_out=""

        with lock:

            for uid,cnt in alert_counter.items():

msg_out+=f"UID:{uid}\nDenied\nCount:{cnt}\n\n"

        send(msg_out or "No alerts");continue

    if clean=="connection":

        if wifi_clients:

            d=wifi_clients[0]

            send(f"MAC:{d.get('mac')}\nRSSI:{d.get('rssi')}")

        else:send("No device")

        continue

    if clean=="count":

        msg_out=""

        for i,d in enumerate(wifi_clients,1):

            msg_out+=f"{i}: {d.get('mac')}
RSSI:{d.get('rssi')}\n"

```

```

        send(msg_out or "No devices");continue

    if lower.startswith("block "):

        uid=text.split()[1].upper()

        blocked.add(uid);send(f"Blocked:{uid}");continue

    if lower.startswith("unblock "):

        uid=text.split()[1].upper()

        blocked.discard(uid)

        uid_count[uid]=0;ok_counter[uid]=0

        send(f"Unblocked:{uid}");continue

    send("Invalid command")

except Exception as e:

    print("Telegram error:",e);time.sleep(1)

if __name__=="__main__":

    print("IDS IoT System Running")

    threading.Thread(target=mqtt_loop,daemon=True).start()

    threading.Thread(target=tg_loop,daemon=True).start()

    while True:time.sleep(1)

```

4.5.3 MQTT command

The MQTT commands are used to set up and monitor communication using the Eclipse Mosquitto broker. First, the command `cd D:\Mosquitto` is used to navigate to the directory where Mosquitto is installed. The command `.\mosquitto.exe -c .\mosquitto.conf -v` starts the MQTT broker using a configuration file, and the `-v` (verbose) option displays real-time logs of connections and message activity. If the broker is already running as a background service, the command `net stop mosquitto` is used to stop it before starting a new instance. Finally, the command `.\mosquitto_sub.exe -h 10.70.40.202 -t rfid/scan -v` is used to subscribe to the topic

rfid/scan on the MQTT broker at the specified IP address, allowing the user to view incoming messages along with their topics. These commands together help in running the MQTT broker and observing real-time data transmission in an IoT system.

It is mqtt subscriber and publisher command

```
cd D:\Mosquitto
```

```
.\mosquitto.exe -c .\mosquitto.conf -v
```

```
net stop mosquitto
```

```
.\mosquitto_sub.exe -h 10.70.40.202 -t rfid/scan -v
```

4.6 Testing

4.6.1 Terminal Testing at Backend

```
IDS in IoT (ESP32-C5 to RC522)
[ALERT] C3D6AFFC Count:1
[OK] Rishabh (E336D9FC) Count:1
[ALERT] C3D6AFFC Count:2
[ALERT] C3D6AFFC Count:3
[OK] Rishabh (E336D9FC) Count:2
[ALERT] C3D6AFFC Count:4
[OK] Rishabh (E336D9FC) Count:3
[OK] Rishabh (E336D9FC) Count:4
[OK] Rishabh (E336D9FC) Count:5
[OK] Rishabh (E336D9FC) Count:6
[OK] Rishabh (E336D9FC) Count:7
[OK] Rishabh (E336D9FC) Count:8
[ALERT] C3D6AFFC Count:5
[OK] Rishabh (E336D9FC) Count:9
Telegram error: HTTPSConnectionPool(host='api.telegram.org', port=443): Read timed out. (read timeout=3)
[OK] Rishabh (E336D9FC) Count:10
[OK] Rishabh (E336D9FC) Count:11
```

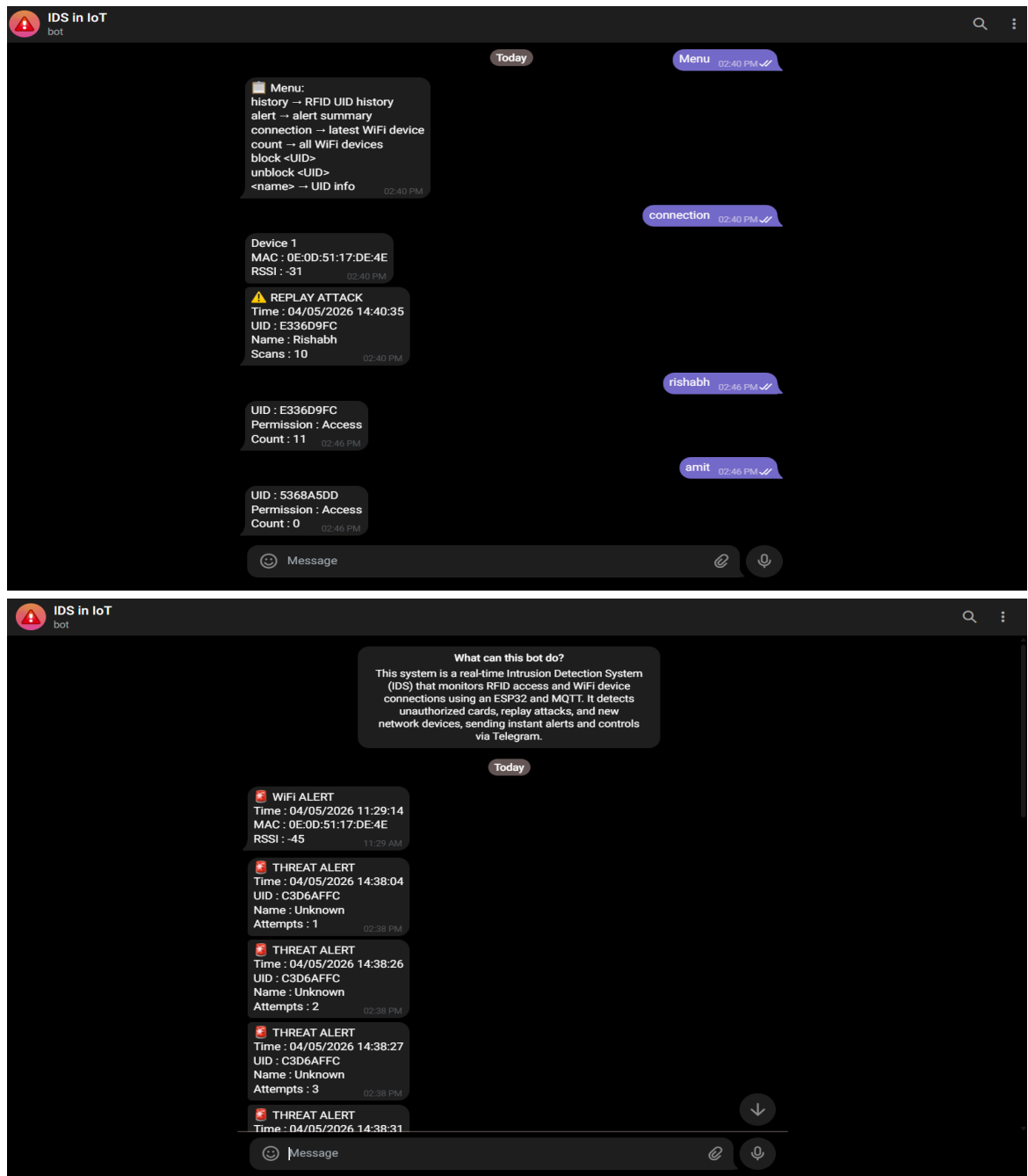
The terminal output you are observing represents backend testing of the IoT-based Intrusion Detection System (IDS) where the ESP32-C5 sends RFID scan data through MQTT and the backend (Python + Telegram system) processes it in real time.

In the logs, each line corresponds to an RFID scan event received from the system. When a known UID such as Rishabh (E336D9FC) is detected, it is marked as [OK], indicating authorized access, and the count increases with each scan. This count is used to track repeated scans and detect abnormal behavior such as replay attacks. On the other hand, unknown or unauthorized UIDs such as C3D6AFFC are marked as [ALERT], which means the system has detected a potential threat. As the number of attempts increases (Count: 1, 2, 3...), the backend logic evaluates whether the activity is suspicious, and after a threshold (e.g., multiple attempts), it can trigger alerts or block the UID.

The increasing count values demonstrate that the backend is correctly maintaining state (tracking how many times a card is scanned). The presence of both [OK] and [ALERT] logs confirms that the system is successfully distinguishing between authorized and unauthorized users. Additionally, the message Telegram error: HTTPSConnectionPool... Read timed out indicates that the backend attempted to send a notification via the Telegram API but failed due to a network timeout, which is a common issue during testing and does not affect the core detection logic.

Overall, this terminal output verifies that the backend system is functioning correctly: it is receiving MQTT messages, processing RFID data, classifying access events, maintaining counters, and attempting to send alerts. This confirms successful integration of ESP32-C5, MQTT communication, and backend monitoring logic in the IoT-based IDS.

4.6.2 Telegram Notification and Chatbot Testing



The Telegram-based interface integrated into the IoT Intrusion Detection System using the ESP32-C5. The Telegram bot acts as a real-time notification and control system, where alerts generated from RFID scans and WiFi monitoring are instantly sent to the user. As shown, the system correctly identifies unauthorized RFID access and generates THREAT ALERT messages with UID, timestamp, and attempt count, while also detecting new WiFi devices and issuing WiFi ALERTS. Additionally, the chatbot functionality allows the administrator to interact with the system using commands such as menu, connection, history, and user names (e.g., “rishabh”, “amit”), which return relevant system data like UID details, access status, and scan counts. The detection of a REPLAY ATTACK confirms that the backend logic is effectively tracking repeated scans and identifying suspicious behavior. Overall, the Telegram testing verifies that the system provides real-time monitoring, alert generation, and remote control capabilities, ensuring efficient and secure IoT system management

5. Conclusion

1. This project successfully designed and implemented an IoT-based Intrusion Detection System (IDS) that integrates hardware, communication protocols, and intelligent monitoring techniques.
2. The system effectively combines RFID-based authentication, MQTT communication, and machine learning-based intrusion detection, providing a complete end-to-end security solution.
3. The use of the ESP32-C5 microcontroller and RC522 RFID module enables reliable and low-cost physical access control through UID-based authentication.
4. The Raspberry Pi 5 acts as a powerful edge device, handling MQTT communication, data processing, and real-time decision-making efficiently.
5. The MQTT publish–subscribe architecture ensures lightweight, scalable, and real-time communication between IoT devices.
6. The machine learning model (Extra Trees with SMOTE and feature engineering) achieved high accuracy (~99.5%) and improved detection of minority attack classes, making it suitable for real-world deployment.
7. The system effectively detects unauthorized access, repeated scans, and replay attacks, enhancing overall security beyond basic authentication mechanisms.
8. Integration with a Telegram bot provides real-time alerts and remote control, improving usability and enabling administrators to monitor the system from anywhere.
9. The adversarial robustness evaluation demonstrates that the system is resistant to most attack strategies, though some vulnerability exists to advanced decision-based attacks like Boundary Attack.
10. The solution is cost-effective, scalable, and suitable for applications such as smart homes, offices, and industrial security systems.

11. The project bridges the gap between hardware-level security and intelligent The implementation ensures software-based intrusion detection, offering a multi-layered security approach.
12. Future improvements can include enhanced adversarial defense techniques, cloud integration, and deployment on real-time large-scale IoT environments.
13. The system demonstrates modular architecture design, allowing easy integration of additional sensors, authentication methods, or security layers without major changes to the existing system.
14. real-time data processing with low latency, making it suitable for critical applications where immediate threat detection and response are required.
15. The project provides a practical prototype for smart security systems, showing how embedded systems, edge computing, and AI can be combined effectively for next-generation IoT security solutions.

6. References

- [1] B. Anthi, L. Williams, M. Słowinska, G. Theodorakopoulos, and P. Burnap, "A supervised intrusion detection system for smart home IoT devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 9042–9053, Oct. 2019.
- [2] D. Boughaci, A. I. Bouameur, and A. Zereg, "An intelligent intrusion detection system for the Internet of Things (IoT) ecosystem," in *Proc. IEEE ICCED*, 2024.
- [3] A. Boughaci and K. Al-Taharwa, "Intrusion detection for IoT networks: A systematic literature review," *IEEE Access*, vol. 11, 2023.
- [4] S. A. Abdulkareem, C. H. Foh, M. Shojafar, F. Carrez, and K. Moessner, "Network intrusion detection: An IoT and non IoT-related survey," *IEEE Access*, vol. 12, pp. 147167–147190, 2024.
- [5] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, 2023.
- [6] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.
- [7] V. Mishra et al., "The use of MQTT in M2M and IoT systems: A survey," *IEEE Access*, vol. 8, pp. 201071–201086, 2020.
- [8] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *Journal of Open Source Software*, vol. 2, no. 13, 2017.
- [9] Eclipse Foundation, "Eclipse Paho - MQTT Client Library for Python," 2026.
- [10] NXP Semiconductors, "MFRC522 standard performance MIFARE and NTAG frontend," *Data Sheet*, Apr. 2016.

- [11] ISO/IEC, "ISO/IEC 14443-1:2018 cards and security devices for personal identification — Contactless proximity objects," International Organization for Standardization, 2018.
- [12] Raspberry Pi Ltd, "Raspberry Pi 5 Model B technical data," Jul. 2025.
- [13] Espressif Systems, "ESP32-C5 series datasheet: Ultra-low-power SoC with 32-bit RISC-V single-core microprocessor," v1.1, Sep. 2025.
- [14] Espressif Systems, "ESP-IDF programming guide: Get started with ESP32-C5," v5.3, Oct. 2025.
- [15] Espressif Systems, "Security features in ESP32 series: Secure boot and flash encryption," Whitepaper, 2024.
- [16] A. Rahman et al., "Scalable machine learning-based intrusion detection system for IoT-enabled smart cities," *Sustainable Cities and Society*, vol. 61, p. 102324, 2020.
- [17] N. Sarwar, I. S. Bajwa, M. Z. Hussain, M. Ibrahim, and K. Saleem, "IoT network anomaly detection in smart homes using machine learning," *IEEE Access*, vol. 11, pp. 119462–119480, 2023.
- [18] J. G. A. Jaramillo-Alcazar, J. Govea, and W. Villegas-Ch, "Anomaly detection in a smart industrial machinery plant using IoT and machine learning," *Sensors*, vol. 23, no. 19, p. 8286, 2023.
- [19] M. S. Mahdavinejad et al., "Machine learning for Internet of Things data analysis: A survey," *Digital Communications and Networks*, vol. 4, no. 3, pp. 161–175, 2018.
- [20] J. Lin et al., "A survey on IoT security: Management and systems," *IEEE Internet of Things Journal*, vol. 4, no. 6, pp. 2025–2058, 2017.
- [21] R. Hummen et al., "Delegation-based authentication and authorization for the IP-based Internet of Things," in *Proc. IEEE TrustCom*, 2012.

- [22] P. Gope and R. Amin, "A novel biometric-based remote user authentication scheme for connected health care," *IEEE Journal of Biomedical and Health Informatics*, vol. 22, no. 4, pp. 1251–1258, 2018.
- [23] M. B. Mollah et al., "Cybersecurity for smart grid: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 1115–1157, 2021.
- [24] K. K. R. Choo, "The cyber threat landscape: Challenges and future directions," *IEEE Cloud Computing*, vol. 1, no. 1, pp. 71–76, May 2014.
- [25] S. K. Patel, "Improving intrusion detection in cloud-based healthcare using neural network," *Biomedical Signal Processing and Control*, vol. 83, p. 104680, 2023.
- [26] O. D. Okey et al., "IDS-MA: Intrusion detection system for IoT MQTT attacks using centralized and federated learning," *IEEE Access*, vol. 11, pp. 1023–1038, 2023.
- [27] M. Akhi, C. Eising, and L. L. Dhirani, "Securing IoT using lightweight TCN for edge deployment on Raspberry Pi 4," *IEEE Open Journal of the Communications Society*, vol. 7, pp. 448–462, Jan. 2026.
- [28] A. Rahman et al., "A systematic review for evaluating IoT security: A focus on authentication, protocols and enabling technologies," *IEEE Xplore*, Feb. 2025.
- [29] R. Jain, "IoT-Based Intrusion Detection System (Minor-II Project)," GitHub repository, Available: <https://github.com/RISHABH12005/Minor-II>

PERSONAL DETAILS

NAME: Ranjan Mishra

ENROLLMENT NUMBER: 231B258

EMAIL ID: ranjanmishra2028@gmail.com

ADDRESS: plot: 413/3901, Dream House, Bhubaneswar, Odisha



NAME: Rishabh Jain

ENROLLMENT NUMBER: 231B264

EMAIL ID: 2r10j5@gmail.com

ADDRESS: Maharaj Bada, Lashkar, Gwalior, Madhya Pradesh



NAME: Rashmi Dubey

ENROLLMENT NUMBER: 231b260

EMAIL ID: rashmi.dubey1010@gmail.com

ADDRESS: Village -Toosa , Singrauli, Madhya Pradesh

